

# POCO4A

# PROGRAMAÇÃO ORIENTADA A OBJETOS

Aula 07- Padrões de Projeto

Prof. Rafael G. **Mantovani**

# Roteiro



- 1** Introdução
- 2** Padrões de Projeto
- 3** Exemplo
- 4** Exercícios
- 5** Referências

# Roteiro

- 1** Introdução
- 2** Padrões de Projeto
- 3** Exemplo
- 4** Exercícios
- 5** Referências

# Introdução

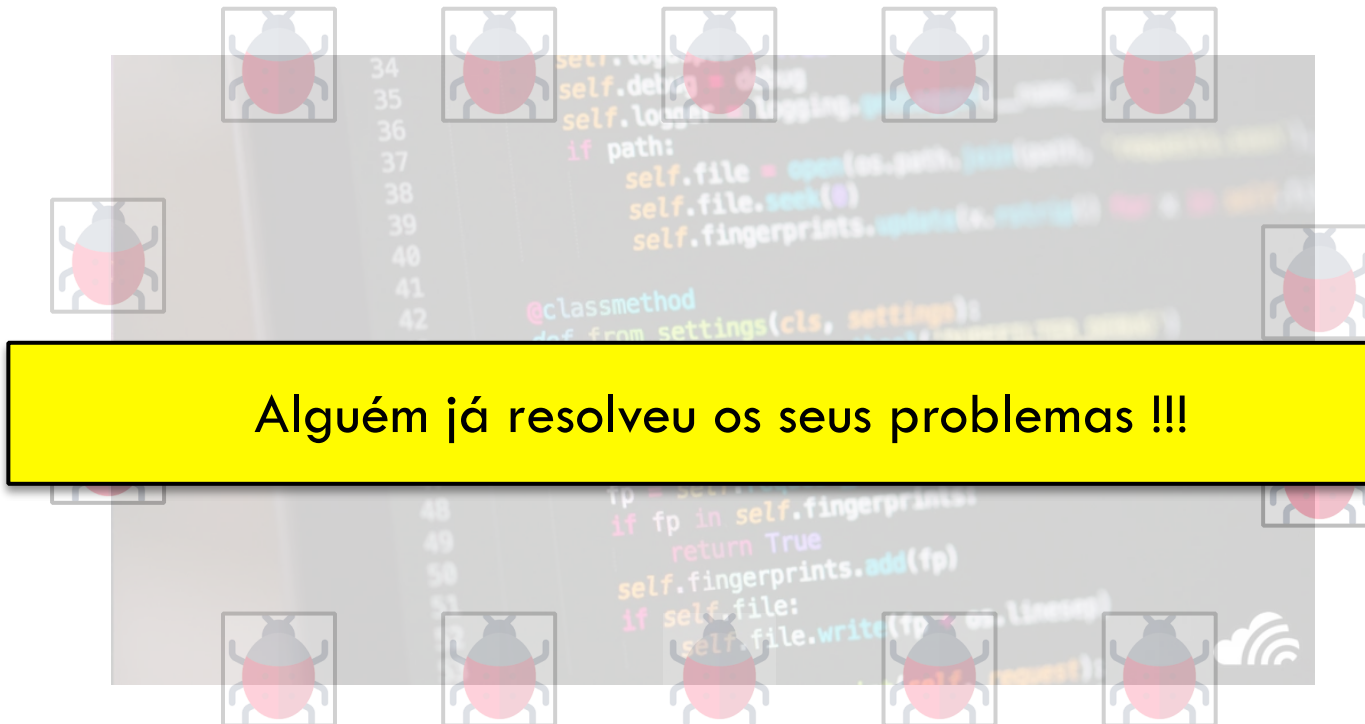
```
34 self.logger = logging.getLogger(__name__)
35 self.debug = debug
36 self.logger = logging.getLogger(__name__)
37 if path:
38     self.file = open(os.path.join(path, 'requests.log'), 'a')
39     self.file.seek(0)
40     self.fingerprints.update({x:request for x in self})
41
42 @classmethod
43 def from_settings(cls, settings):
44     debug = settings.getbool('SUPERMAN_DEBUG')
45     return cls(job_dir(settings), debug)
46
47 def request_seen(self, request):
48     fp = self.request_fingerprint(request)
49     if fp in self.fingerprints:
50         return True
51     self.fingerprints.add(fp)
52     if self.file:
53         self.file.write(fp + os.linesep)
```



# Introdução



# Introdução



# Introdução



**Problemas recorrentes:** sistemas OO frequentemente enfrentam os mesmos problemas de acoplamento, extensibilidade e manutenção



**Experiência utilizável:** programadores experientes reutilizam soluções arquiteturais já validadas em vez de reinventar estruturas de design



**Flexibilidade de software:** padrões ajudam a criar sistemas mais descolados, reutilizáveis e preparados para mudanças

# Introdução



**Programadores  
Experientes**



# Introdução



**Programadores  
Experientes**

- Não resolvem problemas a partir de códigos simples
- Reusam soluções do passado que já funcionaram
- Recorrem a padrões de classes e comunicação de objetos
- Padrões que resolvem problemas, e tornam a OO mais flexível, elegante e reutilizável

# Introdução

**Exemplo:** roteirista



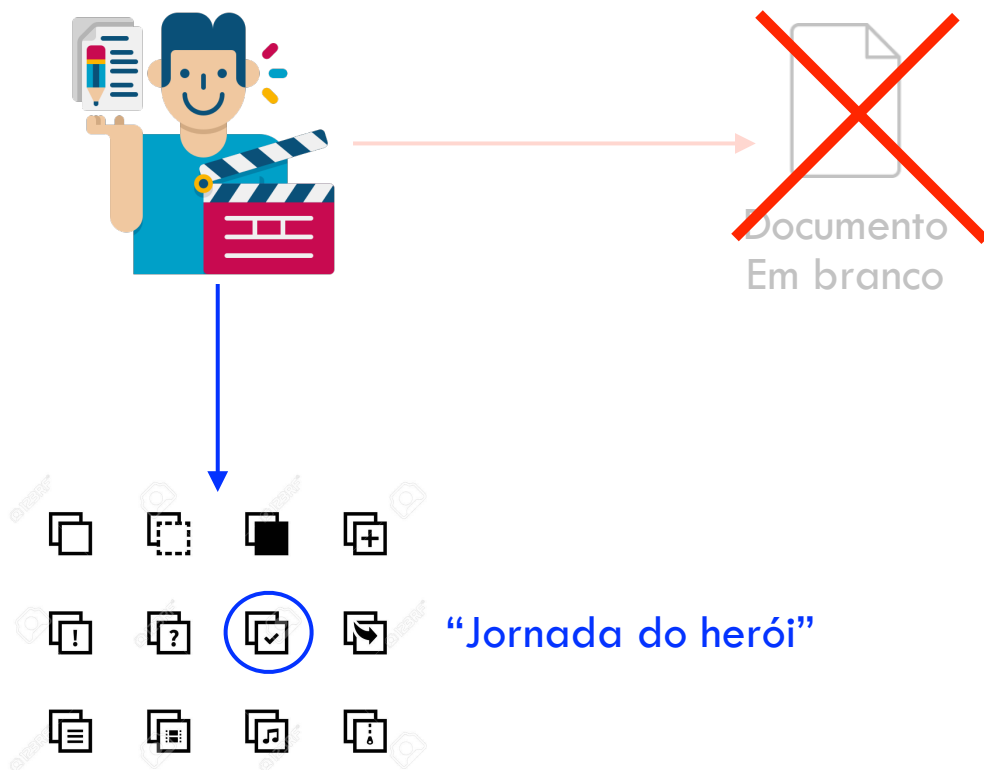
# Introdução

**Exemplo:** roteirista



# Introdução

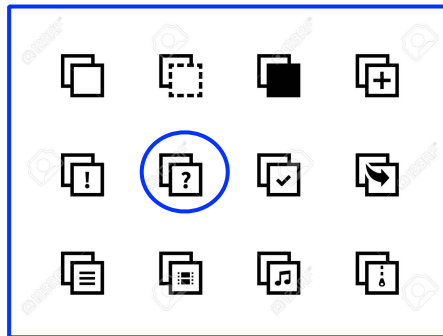
## Exemplo: roteirista



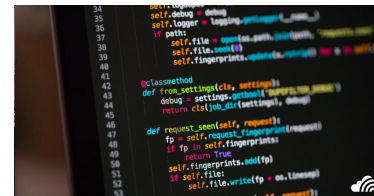
# Introdução

## Exemplo: desenvolvedor

### Padrões de Projeto



"Strategy"



# Introdução

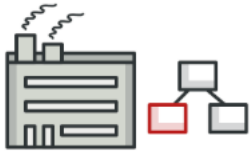
**Objetivo** da aula: compreender padrões de projeto como soluções reutilizáveis para problemas recorrentes.

# Roteiro



- 1 Introdução
- 2 Padrões de Projeto
- 3 Exemplo
- 4 Exercícios
- 5 Referências

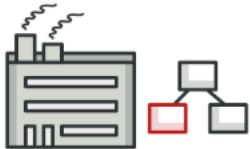
# Padrões de Projeto



**Padrões de projeto**



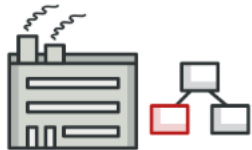
# Padrões de Projeto



## Padrões de projeto

... são **soluções típicas** para problemas comuns em projeto de software. Eles são como **plantas de obra** pré fabricadas que você pode **customizar** para resolver um problema de projeto recorrente em seu código.

# Padrões de Projeto



## Padrões de projeto

... são **soluções típicas** para problemas comuns em projeto de software. Eles são como **plantas de obra** pré fabricadas que você pode **customizar** para resolver um problema de projeto recorrente em seu código.

Padrões são conceitos gerais, não códigos prontos!

# Padrões de Projeto



No geral, um padrão tem:

# Padrões de Projeto

No geral, um padrão tem:

1

2

3

4

# Padrões de Projeto

No geral, um padrão tem:

**1**

**Nome**

**2**

**Problema**

**3**

**Solução**

**4**

**Consequências**

# Padrões de Projeto

No geral, um padrão tem:

- 1 Nome:** descreve o tipo de problema manipulado e sua solução
- 2 Problema:** descreve quando aplicar o padrão
- 3 Solução:** descreve os elementos que corrigem o design, suas relações, responsabilidade e colaborações
- 4 Consequências:** são os resultados de se aplicar o padrão, mostrando os custos (tempo e espaço) e benefícios de se aplicar o padrão

# Padrões de Projeto

**Tabela 1.1** O espaço dos padrões de projeto

| Escopo | Classe        | Propósito                                                                    |                                                                                                                                |                                                                                                                                                                         |
|--------|---------------|------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|        |               | De criação                                                                   | Estrutural                                                                                                                     | Comportamental                                                                                                                                                          |
|        |               | Factory Method (112)                                                         | Adapter (class) (140)                                                                                                          | Interpreter (231)<br>Template Method (301)                                                                                                                              |
|        | <b>Objeto</b> | Abstract Factory (95)<br>Builder (104)<br>Prototype (121)<br>Singleton (130) | Adapter (object) (140)<br>Bridge (151)<br>Composite (160)<br>Decorator (170)<br>Façade (179)<br>Flyweight (187)<br>Proxy (198) | Chain of Responsibility (212)<br>Command (222)<br>Iterator (244)<br>Mediator (257)<br>Memento (266)<br>Observer (274)<br>State (284)<br>Strategy (292)<br>Visitor (305) |

**Fonte:** [Gamma et al, 2000]

# Padrões de Projeto

Tabela 1.1 O espaço dos padrões de projeto

|                                                                              |
|------------------------------------------------------------------------------|
|                                                                              |
| <b>De criação</b>                                                            |
| Factory Method (112)                                                         |
| Abstract Factory (95)<br>Builder (104)<br>Prototype (121)<br>Singleton (130) |

- **Padrões de criação:** estão relacionados ao processo de criação de objetos, encapsulando detalhes de instanciação e promovendo flexibilidade estrutural



# Padrões de Projeto

Tabela 1.1 O espaço dos padrões de projeto

| Propósito              |
|------------------------|
| Estrutural             |
| Adapter (class) (140)  |
| Adapter (object) (140) |
| Bridge (151)           |
| Composite (160)        |
| Decorator (170)        |
| Façade (179)           |
| Flyweight (187)        |
| Proxy (198)            |

- **Padrões estruturais:** tratam da composição de classes e objetos, organizando relacionamentos e reduzindo acoplamento entre componentes

# Padrões de Projeto

Tabela 1.1 O espaço dos padrões de projeto

- **Padrões comportamentais:** focam na comunicação entre objetos e distribuição de responsabilidades dentro do sistema

| Comportamental                |
|-------------------------------|
| Interpreter (231)             |
| Template Method (301)         |
| Chain of Responsibility (212) |
| Command (222)                 |
| Iterator (244)                |
| Mediator (257)                |
| Memento (266)                 |
| Observer (274)                |
| State (284)                   |
| Strategy (292)                |
| Visitor (305)                 |

# Padrões de Projeto

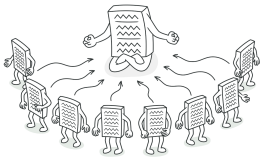
Tabela 1.1 O espaço dos padrões de projeto

|        |        | Propósito            |                       |                                                |
|--------|--------|----------------------|-----------------------|------------------------------------------------|
|        |        | De criação           | Estrutural            | Comportamental                                 |
| Escopo | Classe | Factory Method (112) | Adapter (class) (140) | Interpreter (231)<br>Template Method (301)     |
|        |        |                      |                       | (212)                                          |
|        |        |                      | Proxy (198)           | State (284)<br>Strategy (292)<br>Visitor (305) |

**Escopo de Classe** vs **Escopo de objeto**: alguns padrões atuam por herança entre classes, enquanto a maioria utiliza composição e colaboração entre objetos

Fonte: [Gamma et al, 2000]

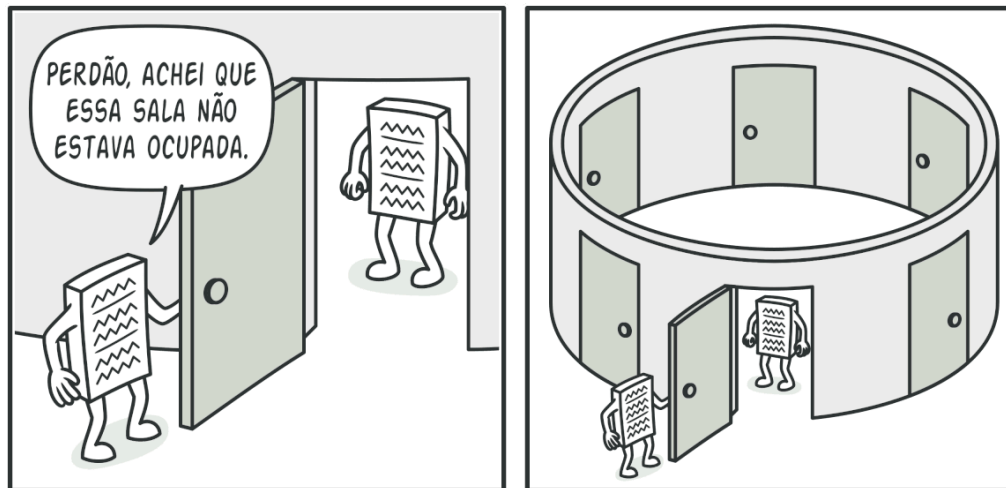
# Exemplo: Singleton



## Singleton

**Intenção:** garantir que uma classe tenha apenas uma instância, enquanto provê um ponto de acesso global para essa instância.

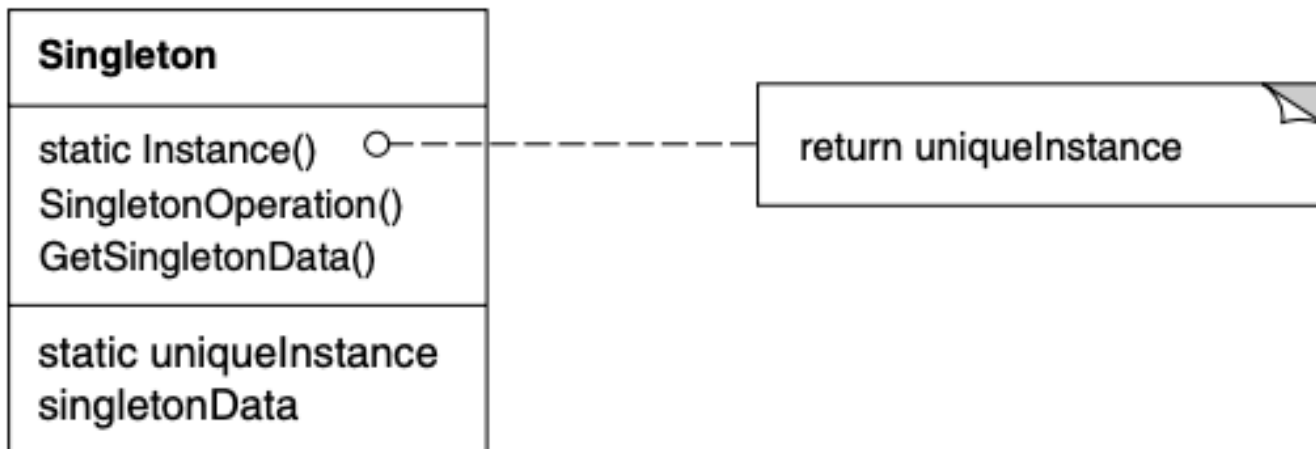
**Motivação:** controlar o acesso de algum recurso compartilhado, como uma base de dados ou arquivo.



# Exemplo: Singleton

**Aplicabilidade:** usar quando for preciso haver uma instância da classe, e essa instância tiver que dar acesso aos clientes através de um ponto bem conhecido

## Estrutura:



**Fonte:** [Gamma et al, 2000]

# Exemplo: Singleton



C++

```
1. class Singleton{
2.     public:
3.         static Singleton* Instance();
4.     protected:
5.         Singleton();
6.     private:
7.         static Singleton* _instance;
8. };
9.
10. Singleton* Singleton::_instance = 0;
11.
12. Singleton* Singleton::Instance() {
13.     if(_instance == 0) {
14.         _instance = new Singleton;
15.     }
16.     return _instance;
17. }
```

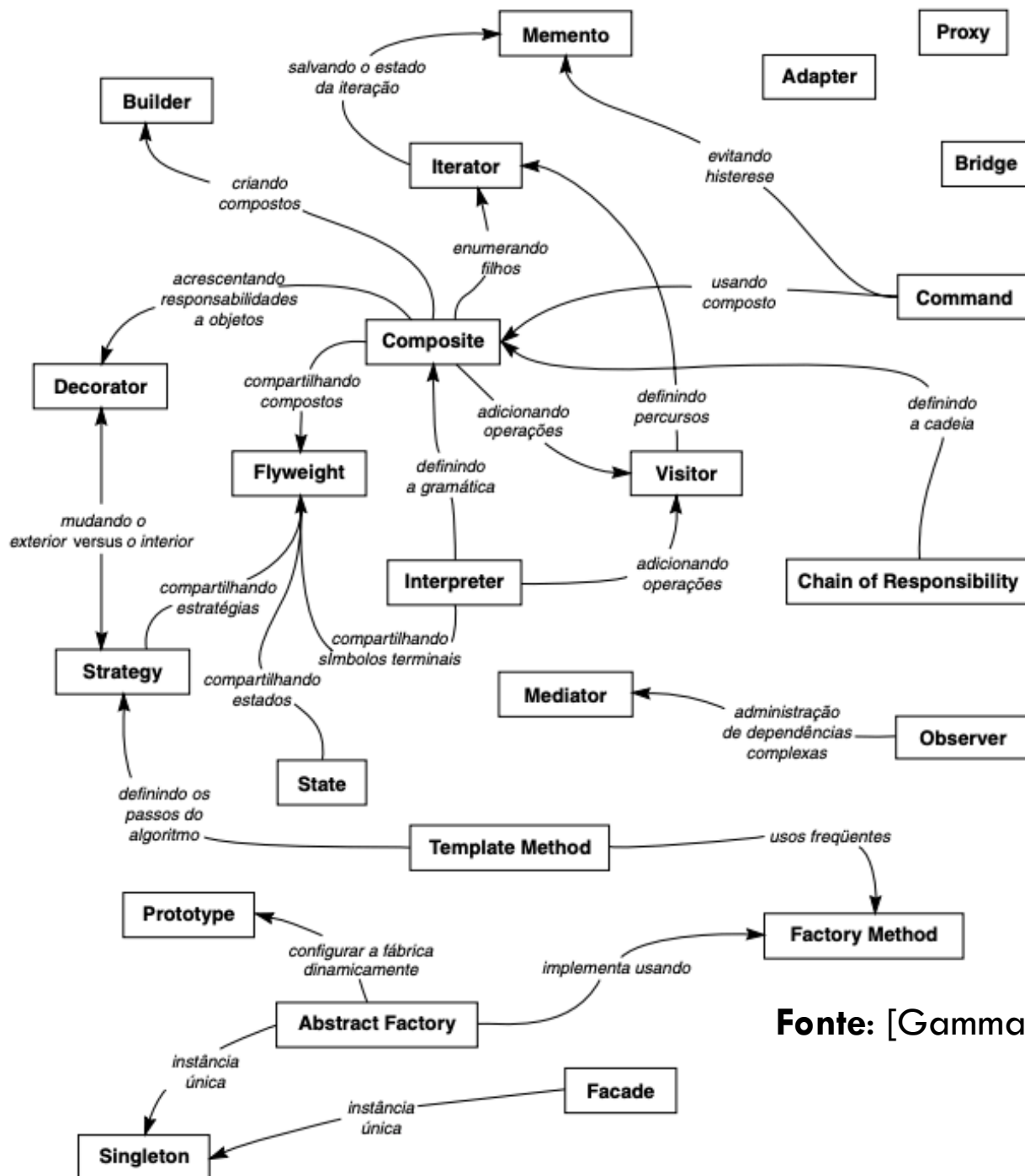
# Padrões de Projeto

- Alguns padrões costumam ser usados juntos:
  - **Composite** é frequentemente usado com **Iterator** ou **Visitor**
- Outros são alternativas ao mesmo problema:
  - **Prototype** é uma alternativa para o **Abstract Factory**

# Padrões de Projeto

- Alguns padrões costumam ser usados juntos:
    - **Composite** é frequentemente usado com **Iterator** ou **Visitor**
  - Outros são alternativas ao mesmo problema:
    - **Prototype** é uma alternativa para o **Abstract Factory**
- **Programar** para uma **interface**, não uma implementação
  - Prefira a **composição** de objeto à herança de classe (delegação)





Fonte: [Gamma et al, 2000]

# Padrões de Projeto

Tabela 1.1 O espaço dos padrões de projeto

|        |        | Propósito                                                                                  |                                                                                                                                |                                                                                                                                                                                       |
|--------|--------|--------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|        |        | De criação                                                                                 | Estrutural                                                                                                                     | Comportamental                                                                                                                                                                        |
| Escopo | Classe | <b>Factory Method (112)</b>                                                                | Adapter (class) (140)                                                                                                          | Interpreter (231)<br>Template Method (301)                                                                                                                                            |
|        | Objeto | <b>Abstract Factory (95)</b><br>Builder (104)<br>Prototype (121)<br><b>Singleton (130)</b> | Adapter (object) (140)<br>Bridge (151)<br>Composite (160)<br>Decorator (170)<br>Façade (179)<br>Flyweight (187)<br>Proxy (198) | Chain of Responsibility (212)<br>Command (222)<br>Iterator (244)<br>Mediator (257)<br>Memento (266)<br>Observer (274)<br>State (284)<br><b>Strategy (292)</b><br><b>Visitor (305)</b> |

Fonte: [Gamma et al, 2000]

# Roteiro

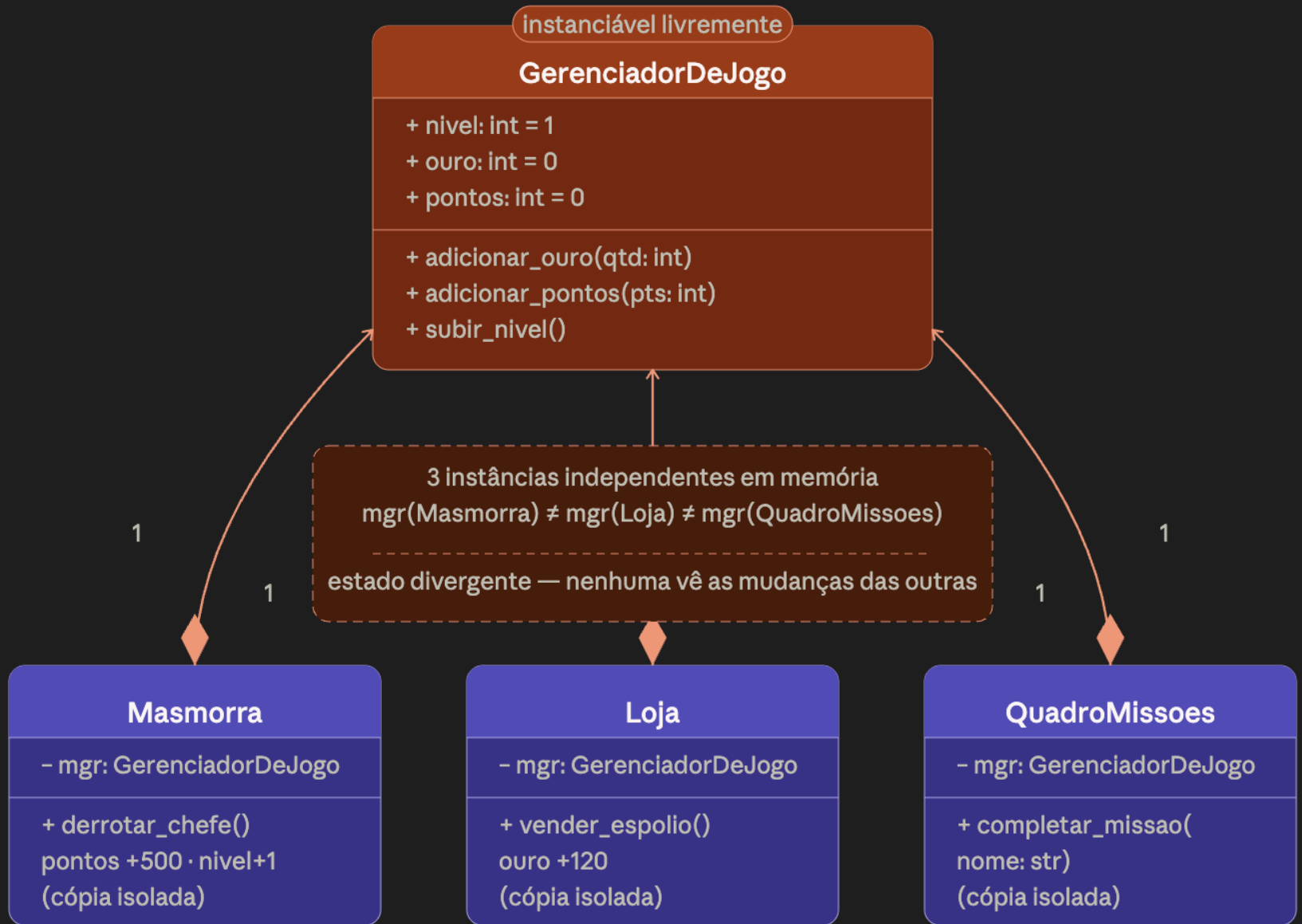


- 1 Introdução
- 2 Padrões de Projeto
- 3 Exemplo
- 4 Exercícios
- 5 Referências

# Exemplo

## A Guilda do Único — estado global de jogo

- Em Crônicas de Eldoria, três sistemas independentes — a Masmorra, a Loja e o Quadro de Missões — precisam acessar e modificar o mesmo estado global: nível, ouro e pontuação do jogador.
- Atualmente cada sistema cria sua própria instância de **GerenciadorDeJogo**. Isso causa um bug crítico: ao derrotar o chefe da masmorra, a loja e o quadro de missões continuam lendo seus próprios objetos zerados, como se nada tivesse acontecido.



◆ → composição (tem-um)

----- anotação de problema

instanciável livremente

# Exemplo

## Relatório de Erro (bug observado)

- ❑ Masmorra derrota o chefe → nível 2, pontuação +500
- ❑ Loja acessa o gerenciador → ainda vê nível 1, pontuação 0
- ❑ Missão completa → ouro somado à cópia da missão, masmorra nunca vê
- ❑ Tela final exibe 0 de ouro e nível 1 — estado perdido

# Exemplo



## Python

```
1. class GerenciadorDeJogo:
2.     def __init__(self, name):
3.         self.nivel = 1
4.         self.ouro = 0
5.         self.pontos = 0
6.
7.     def adicionar_ouro(self, qtd: int):
8.         self.ouro += qtd
9.
10.    def adicionar_pontos(self, pts: int):
11.        self.pontos += pts
12.
13.    def subir_nivel(self):
14.        self.nivel += 1
15.
16.    def exibir(self):
17.        print(f"Gerenciador id={id(self)} "f"nivel={self.nivel}"
18.              ouro={self.ouro} pontos={self.pontos}")
```

# Exemplo



## Python

```
1. class Masmorra:
2.     def __init__(self):
3.         self.mgr = GerenciadorDeJogo()
4.         self.ouro = 0
5.         self.pontos = 0
6.
7.     def derrotar_chefe(self):
8.         self.mgr.adicionar_pontos(500)
9.         self.mgr.subir_nivel()
10.        print("[Masmorra] Chefe derrotado!")
11.        self.msg.exibir()
```



# Exemplo



## Python

```
1. class Loja:
2.     def __init__(self):
3.         self.mgr = GerenciadorDeJogo()
4.         self.ouro = 0
5.         self.pontos = 0
6.
7.     def vender_espolio(self):
8.         self.mgr.adicionar_ouros(120)
9.         print("[Loja] Espólio vendido!")
10.        self.msg.exibir()
11.
```

# Exemplo



## Python

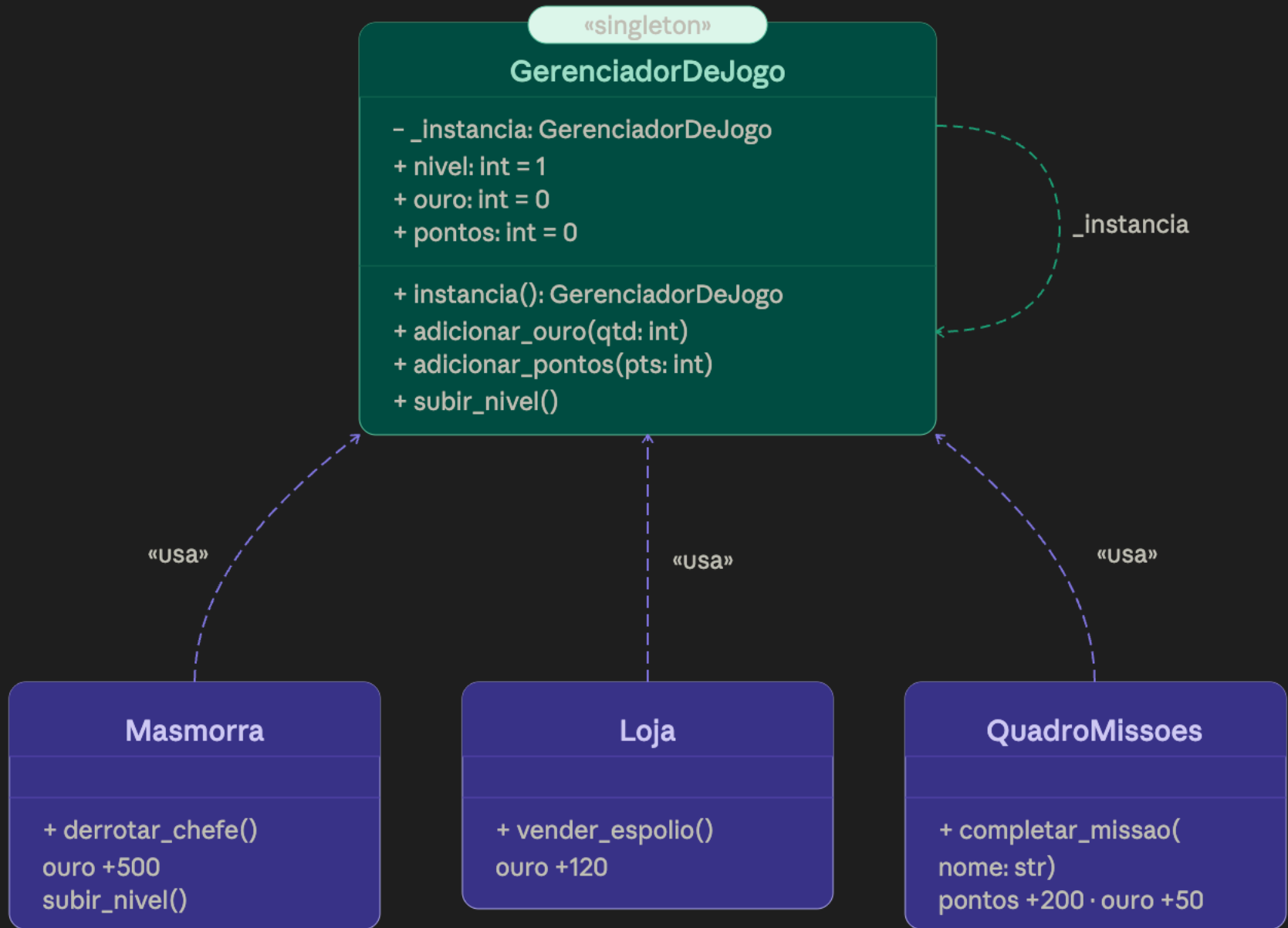
```
1. if __name__ == "__main__":  
2.     masmorra = Masmorra()  
3.     loja      = Loja()  
4.  
5.     masmorra.derrotar_chefe()  
6.     loja.vender_espolio()    # estado divergente!  
7.
```

# Exemplo



## Objetivos

- Identificar o **problema** causado por múltiplas instâncias de uma classe com papel conceitual único
- Implementar o **Singleton** em **Python/C++**
- Verificar que duas chamadas retornam o mesmo objeto



---> dependência («usa»)

---> auto-referência (\_instancia)

«singleton» estereótipo

# Roteiro



- 1 Introdução
- 2 Padrões de Projeto
- 3 Exemplo
- 4 Exercícios
- 5 Referências

# Exercícios



**HANDS ON :)))**

# Exercício 01

- Em *Crônicas de Eldoria*, eventos de batalha são registrados por diversas classes — o herói, os inimigos e o sistema de magia. Cada uma cria seu próprio **Logger**, gerando arquivos de log duplicados e mensagens fora de ordem.
- **O QUE IMPLEMENTAR:**
  - classe **Logger** como **Singleton** com método `registrar(msg)`
  - classe **Heroi** com método `atacar(alvo)` — registra o ataque
  - classe **Inimigo** com método `receber_dano(qtd)` — registra o dano
  - ao final, exibir todos os logs com `Logger.exibir_todos()`
  - provar que `Logger()` chamado duas vezes retorna o mesmo objeto

# Exercício 01

## Exemplo de saída:

```
[LOG #1] Herói atacou Dragão com espada  
[LOG #2] Dragão recebeu 40 de dano  
[LOG #3] Herói usou Bola de Fogo em Dragão  
[LOG #4] Dragão recebeu 80 de dano
```

Total de eventos: 4

Mesma instância? True ✓

**Restrição:** `Logger` não pode ser instanciado diretamente mais de uma vez. O construtor deve ser protegido. A lista de mensagens pertence à instância única.



# Exercício 02

- O jogo possui configurações como dificuldade, volume e idioma. O menu principal altera essas configurações e a masmorra precisa lê-las. Sem **Singleton**, a masmorra lê valores padrão antigos porque cada módulo criava seu próprio objeto de configuração.
- **O QUE IMPLEMENTAR:**
  - classe **Configuracao** como **Singleton** com atributos: dificuldade, volume, idioma
  - método **alterar(chave, valor)** para modificar qualquer configuração
  - método **exibir()** para listar todas as configurações atuais
  - classe **MenuPrincipal** que altera dificuldade e volume
  - classe **Masmorra** que lê e exibe as configurações sem alterá-las
  - Provar que os dois módulos acessam exatamente o mesmo objeto

# Exercício 02

## Exemplo de saída:

```
[Menu] Configurações alteradas pelo jogador
[Config] dificuldade = dificil
[Config] volume = 60
[Config] idioma = pt-BR

[Masmorra] Lendo configurações do jogo...
[Config] dificuldade = dificil
[Config] volume = 60
[Config] idioma = pt-BR

Mesma instância? True ✓
```

**Restrição:** `Masmorra` não recebe a configuração por parâmetro — ela obtém a instância diretamente via `Configuracao.instancia()`. Nenhum módulo pode guardar a configuração como atributo de instância.

# Entregável Semanal

- Padrões de Projeto



# Entregável Semanal 07

- **Padrões de Projeto**

**Objetivo:** Analise seu código e identifique pontos que possam melhorados via padrões de projeto

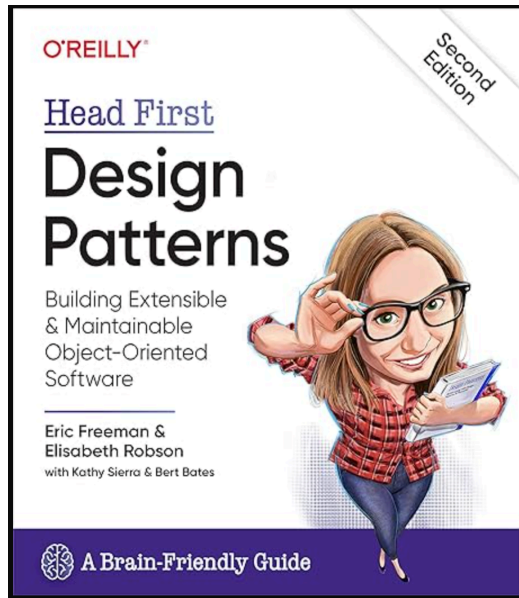


# Roteiro

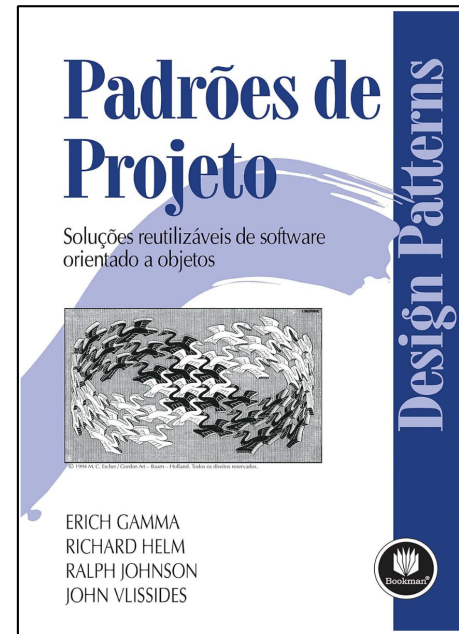


- 1 Introdução
- 2 Padrões de Projeto
- 3 Exemplo
- 4 Exercícios
- 5 Referências

# Referências sugeridas



[Freeman & Robson, 2021]



[Gamma et al, 2000]

# Referências online



[Refactoring Guru]



# Obrigado :)

Prof. Rafael G. **Mantovani**  
[rafaelmantovani@utfpr.edu.br](mailto:rafaelmantovani@utfpr.edu.br)